

MS&E 228 (Winter 2026) — Lecture 11 Notes

CATE Model Selection, Interpretation, and Validation

Vasilis Syrgkanis
Stanford University

February 16, 2026

Readings. *Applied Causal Inference Powered by ML and AI*, §15. Relevant papers: Chernozhukov et al. (2023), Nie & Wager (2021), Foster & Syrgkanis (2023), Dwivedi et al. (2020), Yadlowsky et al. (2025), Ferstad et al. (2024).

Jupyter Notebooks 1. CATE Estimation, Ensembling and Validation, 2. CATE Validation with EconML. 3. DR Learner and Interpretability with EconML.

1 Introduction: From Estimation to a Complete CATE Workflow

In the last lecture, we introduced a suite of powerful *meta-learners* for estimating the Conditional Average Treatment Effect (CATE), $\tau(x) = \mathbb{E}[Y(1) - Y(0)|X = x]$. These methods, including the S-Learner, T-Learner, DR-Learner, and R-Learner, provide a recipe for leveraging any generic machine learning algorithm to estimate how treatment effects vary across individuals. We focused on linear and causal forest variants, which have the advantage of producing confidence intervals for their CATE predictions. However, in many applications, our primary goal is to obtain the most accurate point prediction of the CATE, even if it means sacrificing built-in confidence intervals. This opens the door to using a much wider array of powerful, non-parametric, "black-box" machine learning models like gradient boosting, random forests, and neural networks in the final stage of our meta-learners.

This flexibility, however, presents a new set of challenges. In practice, we are not left with a single CATE estimate, but rather a zoo of potential models arising from different combinations of meta-learners, final-stage ML methods, and hyperparameters. This chapter addresses the crucial questions that arise from this complexity, moving beyond pure estimation to establish a complete, end-to-end workflow for reliable CATE analysis. We will cover four main pillars:

1. **Model Selection:** How should we select the best CATE model from a diverse set of candidates? We will introduce a principled method using the **doubly robust loss (DR-Loss)** as a universal scoring rule. We will see that, in randomized controlled trials (RCTs), this loss serves as a direct, unbiased estimate of the true CATE mean squared error (MSE), making it the ideal metric for model evaluation. While in observational settings it is a robust loss for evaluation.

2. **Interpretation:** How do we interpret what a black-box CATE model has learned? We will explore techniques for understanding and explaining its predictions, including **distillation trees** for a simple, global summary, and **SHAP (SHapley Additive exPlanations)** for granular, per-feature insights.
3. **Statistical Validation:** How do we validate that the heterogeneity captured by our model is real and not just statistical noise? We will develop a battery of statistical tests to perform on a held-out dataset, including the **Best Linear Predictor (BLP)** test, **Group Average Treatment Effects (GATEs)**, and the **Qini curve**.
4. **In-Class Activity:** We will apply this entire workflow to a real-world case study from a recent research paper on digital health interventions, tackling the challenges of observational data and the importance of feature representation.

By the end of this chapter, you will be equipped with a comprehensive toolkit to not only estimate CATEs but also to select, interpret, and rigorously validate your models, paving the way for the deployment of reliable and impactful personalized decision-making systems.

2 CATE Model Selection: A Principled Approach

Having established a rich toolkit of CATE estimators, we face a critical model selection problem. Each of these *meta-learners* (S, T, DR, R, X; see Appendix A for the X-Learner) can be paired with a variety of final-stage machine learning models (e.g., Lasso, Random Forest, XGBoost, Neural Networks), each with its own set of hyperparameters. This creates a vast, combinatorial model space. Beyond the above main meta-learners, there are many other published variations. The central challenge is to navigate this space to identify the single best model for our specific application.

To tackle this, we adopt a principled, two-stage approach:

1. **Hyperparameter Tuning within each Meta-Learner:** For each meta-learner family (e.g., for the DR-Learner), we first need to find the optimal internal ML model and its associated hyperparameters.
2. **Final Model Selection across Meta-Learners:** Once we have the best version of each meta-learner, we need a way to compare them against each other to select the overall champion.

2.1 Step 1: Hyperparameter Tuning via Native Loss Functions

The key insight for the first step is that each meta-learner is itself derived from the minimization of a specific loss function. This *native loss* provides the natural metric for tuning the internal ML model.

Practical Recipe for Hyperparameter Tuning

For a given meta-learner, use its corresponding out-of-sample loss as the evaluation metric for tuning the hyperparameters of its final-stage ML model. This is most effectively done using an automated machine learning (AutoML) framework like FLAML.

For example:

- **DR-Learner:** The final stage of the DR-Learner involves regressing the doubly robust pseudo-outcomes (Y^{DR}) on the covariates X . This is a standard regression problem where the objective is to minimize the Mean Squared Error (MSE). Therefore, we can use a standard AutoML routine with out-of-sample MSE (or R^2) as the metric to tune the final-stage model.
- **R-Learner:** The final stage of the R-Learner minimizes a weighted MSE, where the weights are the squared treatment residuals (the R-Loss). As discussed in the previous chapter, it is crucial to use an **un-normalized** weighted MSE for tuning, as standard weighted MSE metrics in many libraries normalize the weights, which can lead to a mismatch with the R-Learner's theoretical objective, especially during cross-validation. This often requires defining a custom metric within the AutoML framework.

Python Pseudocode: Hyperparameter Tuning with FLAML

```
from flaml import AutoML
from sklearn.base import clone

# --- DR-Learner Tuning ---
# Y_dr_train is computed from first-stage nuisance models
automl_dr = AutoML(task="regression",
                   metric="mse",          # DR-Loss = MSE of predicting Y_dr
                   time_budget=10,        # seconds
                   estimator_list=["xgboost", "rf", "lgbm", "lassolars", "enet"])
automl_dr.fit(X_train, Y_dr_train)
best_dr_model = clone(automl_dr.model)

# --- R-Learner Tuning ---
# Requires a custom, un-normalized weighted MSE metric
def weighted_sse(X_val, y_val, estimator, labels,
                 X_train, y_train, weight_val=None, weight_train=None, *args):
    y_pred = estimator.predict(X_val)
    loss = np.mean(weight_val * (y_val - y_pred) ** 2)
    return loss, {"weighted_sse": loss}

automl_r = AutoML(task="regression", eval_method='cv',
                  metric=weighted_sse, time_budget=10,
                  estimator_list=["xgboost", "rf", "lgbm", "lassolars", "enet"])
# Y_res/D_res are residualized outcome/treatment from first stage
# D_res**2 are the R-Learner weights
automl_r.fit(X_train, Y_res / D_res, sample_weight=D_res**2)
best_r_model = clone(automl_r.model)
```

By following this procedure for each meta-learner, we obtain a set of optimized candidate models: the best-tuned DR-Learner, the best-tuned R-Learner, and so on.

2.2 Step 2: Final Model Selection via the DR-Loss

With a slate of tuned candidate models, $\mathcal{M} = \{\hat{\tau}_1, \hat{\tau}_2, \dots, \hat{\tau}_M\}$, we need a common, fair basis for comparison. The recommended approach is to use the **DR-Loss** as the ultimate arbiter, evaluated on a separate, held-out scoring set.

The DR-Loss as a Universal Scoring Rule

To select the best model from a collection of candidates, $\mathcal{M}$, we choose the one that minimizes the DR-Loss on a held-out scoring set:

$$\hat{\tau}_{\text{best}} = \arg \min_{\hat{\tau} \in \mathcal{M}} \frac{1}{n_{\text{score}}} \sum_{i \in \text{ScoreSet}} (Y_i^{\text{DR}} - \hat{\tau}(X_i))^2$$

For the nuisance models needed to compute the DR pseudo-outcomes on the scoring set, we use models trained on the main training data.

Python Pseudocode: Best Model Selection

```
# Assume:
# - candidate_models = a list of fitted CATE models (e.g., best_dr_model,
#   best_r_model)
# - X_score, Y_score, D_score = scoring data
# - g_hat, e_hat = nuisance models trained on main training data

Y_dr_score = compute_dr_pseudooutcomes(Y_score, D_score, g_hat, e_hat)

best_model = None
min_dr_loss = float('inf')

for model in candidate_models:
    tau_hat_score = model.predict(X_score)
    dr_loss = np.mean((Y_dr_score - tau_hat_score)**2)

    if dr_loss < min_dr_loss:
        min_dr_loss = dr_loss
        best_model = model

# best_model is your final CATE estimator
```

2.3 Why the DR-Loss Is Special

The DR-Loss is the ideal metric for CATE model selection because, under certain conditions, minimizing the DR-Loss is equivalent to minimizing the Mean Squared Error with respect to the true, unobservable CATE function, $\tau(X)$.

Key Theoretical Result (Randomized Experiments)

In a randomized experiment (where the propensity is known), the DR-Loss is a **perfect out-of-sample loss**:

$$\mathbb{E}\left[\left(Y_i^{DR} - \hat{\tau}(X_i)\right)^2\right] = \mathbb{E}\left[\left(\tau(X_i) - \hat{\tau}(X_i)\right)^2\right] + \text{const}$$

It minimizes the RMSE with respect to the **individual treatment effect**—without any assumptions on nuisance models. It can’t get better than that!

In an RCT, we can simply use a constant propensity model (e.g., the overall treatment assignment probability), and the DR-Loss provides an oracle-like model selector.

Technical Note: Observational Studies

In observational settings, the DR-Loss is still a good selector, but with a caveat. The guarantee is that, with high probability, the MSE of the selected model is close to the MSE of the best possible model in our candidate set, up to a term that depends on the quality of our nuisance models:

$$\text{MSE}(\hat{\tau}_{\text{chosen}}) \lesssim \text{MSE}(\hat{\tau}_{\text{best}}) + \sqrt{\frac{\log M}{n}} + \text{RMSE}(\hat{g}) \cdot \text{RMSE}(\hat{p})$$

where M is the number of candidate models and n is the scoring sample size. The term $\text{RMSE}(\hat{g}) \cdot \text{RMSE}(\hat{p})$ highlights the importance of having accurate nuisance models. If this product is small, the DR-Loss is an excellent model selector.

There is a subtle caveat in the above bound: ideally we want to replace the MSE terms in the inequality with RMSE, i.e., we would want a guarantee of the form:

$$\text{RMSE}(\hat{\tau}_{\text{chosen}}) \lesssim \text{RMSE}(\hat{\tau}_{\text{best}}) + \sqrt{\frac{\log M}{n}} + \text{Error}(\hat{g}) \cdot \text{Error}(\hat{p})$$

The earlier bound wouldn’t be implying, for instance, parametric rates if our CATE models were parametric, but only $n^{-1/4}$ rates. This ideal bound would remove this problem. Such an ideal bound does hold for the DR-loss, albeit, for general CATE models, we need to use a slightly stronger error bound for the nuisance models, using mean-fourth-error. For instance, it holds if we use as $\text{Error}(\hat{g}) = \left(\mathbb{E}[(\hat{g}(D, X) - g_0(D, X))^4]\right)^{1/4}$ and similarly for $\hat{p}$. Such fourth moment bounds are slightly stronger requirements than RMSE bounds on the nuisance models.

2.4 Alternative: Stacking CATE Models

Instead of picking one best model, we can also **stack** them: create an ensemble where the final prediction is a weighted average of all candidate model predictions, $\hat{\tau}_{\text{stack}}(x) = \sum_{m=1}^M w_m \hat{\tau}_m(x)$.

Stacking via DR-Loss Minimization

We find the optimal weights $w = (w_1, \dots, w_M)$ by minimizing the DR-Loss of the stacked model on the scoring set. This is a simple least-squares problem:

$$\hat{w} = \arg \min_{w \in W} \frac{1}{n_{\text{score}}} \sum_{i \in \text{ScoreSet}} \left(Y_i^{DR} - \sum_{m=1}^M w_m \hat{\tau}_m(X_i) \right)^2$$

where W is some space of weights. For interpretability, we often constrain the weights to be non-negative and sum to one ($w \in \Delta_M$).

Python Pseudocode: Stacking

```
from sklearn.linear_model import LinearRegression

# Predictions of each candidate model on the scoring set
# Shape: (n_samples, n_models)
all_preds_score = np.array(
    [model.predict(X_score) for model in candidate_models]).T

# Target: DR pseudo-outcomes on the scoring set
Y_dr_score = compute_dr_pseudooutcomes(Y_score, D_score, g_hat, e_hat)

# Fit OLS to find the weights (or use NNLS for simplex constraint)
# Or LassoCV with positive=True, for non-negative coefficients.
# For simplicity, we show unconstrained OLS here
stacker = LinearRegression(fit_intercept=False)
stacker.fit(all_preds_score, Y_dr_score)
stacking_weights = stacker.coef_

# The final stacked model's prediction is a weighted average
# On new data X_val:
all_preds_val = np.array([m.predict(X_val) for m in candidate_models]).T
final_cate_pred = all_preds_val @ stacking_weights
```

2.5 The Complete Data Splitting Strategy

To implement this entire workflow robustly, a three-way data split is recommended:

Recommended Data Split: 60 / 20 / 20

1. **Training set (60%):** Train all meta-learners (nuisance models + final CATE models). Tune hyperparameters within each learner using its native loss (and cross-validation within the training set).
2. **Scoring set (20%):** Compute DR pseudo-outcomes using nuisance models from the training set. Select or stack CATE models via the DR-Loss.
3. **Validation set (20%):** Reserved for the final statistical tests of the chosen CATE model. This data is never touched during any training or selection step.

This disciplined process ensures that our model selection is not biased by overfitting and that our final validation results provide a true assessment of the chosen model’s performance on unseen data.

We now have a “best guess” CATE model. The next steps are to interpret it and validate it.

Running Example: The Welfare Experiment

Throughout the next two sections, we will use the *Welfare Experiment* as a running example. This was a randomized experiment embedded in the General Social Survey (GSS) with $n = 12,907$ respondents and 42 covariates.

- **Treatment (D):** Randomly framing a survey question about government spending using the word **welfare** (treatment) versus the more neutral **assistance to the poor** (control).
- **Outcome (Y):** The respondent’s willingness to support the program, on a numerical scale.
- **Goal:** Estimate the CATE of the word **welfare** on support for the program. The effect is expected to be negative (the word **welfare** has negative connotations for some) and heterogeneous, varying by political ideology, education, income, etc.

3 CATE Interpretation: Understanding Black-Box Models

After selecting our best-performing CATE model, $\hat{\tau}(X)$, we are left with two crucial tasks: interpretation and validation. The selected model is often a complex, *black-box* algorithm like a gradient-boosted forest or a neural network, which provides accurate predictions but offers little direct insight into its decision-making process. We need to answer the fundamental question: What are the key drivers of treatment effect heterogeneity? What are the defining characteristics of the subgroups that are predicted to have high or low treatment effects?

Interpretation in Context

While the CATE predictions can be used directly in fully automated algorithmic systems (e.g., online ad targeting), in most other contexts (e.g., clinical decision support, policy making), the black-box model must be interpreted for a human decision-maker. This is necessary to build trust, ensure fairness, and justify the resulting personalized policy.

All analysis in this section can be performed using the scoring set.

3.1 Interpretation Tool 1: Distillation with a Shallow Tree

A highly effective and simple method for interpreting a complex CATE model is to **distill** its knowledge into a more transparent model. The goal is not to replace the original model, but to create an interpretable approximation that reveals its most important decision logic.

Algorithm: Distillation Tree

Train a shallow `DecisionTreeRegressor` (e.g., with `max_depth=2` or `3`) to predict the CATE model’s own predictions, $\hat{\tau}(X)$, using the original features X . The resulting tree provides a simplified map of how the complex model assigns treatment effects, highlighting the most important features and interaction effects that drive heterogeneity.

In the Welfare Experiment, a depth-2 distillation tree (Figure 5) reveals the primary factors driving heterogeneity. The first and most important split is on `polviews` (political views on a scale from 1=liberal to 7=conservative). For the more left-leaning group, the next most important feature is `educ` (education level). This simple tree suggests that the largest treatment effect heterogeneity is found at the intersection of political ideology and education. Specifically, it identifies that right-leaning, white individuals tend to have the most negative reaction to the word “welfare,” while left-leaning, well-educated individuals also have a negative reaction, but a less pronounced one.

Are these Causal Features?

This is a critical distinction. The features identified by the distillation tree (or SHAP) are **not** necessarily causal moderators of the treatment effect. They are descriptive. The analysis identifies subgroups for which the effect differs, but it does not explain *why* the effect differs. The difference could be due to the feature itself or due to other unobserved factors correlated with that feature. The overall estimated effect is causal, but the source of its heterogeneity is descriptive.

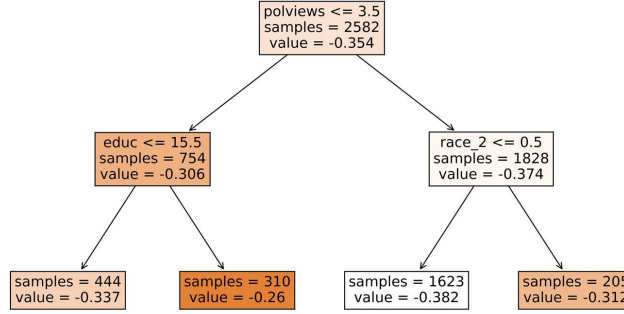


Figure 1: A depth-2 decision tree trained to predict the CATE model’s outputs in the Welfare Experiment. It highlights that political views and education are the primary drivers of heterogeneity. The most negative effect is for right-leaning individuals, while the least negative is for left-leaning, highly educated individuals. See Jupyter Notebook.

Python Pseudocode: EconML’s SingleTreeCateInterpreter

EconML provides a convenient wrapper for this procedure.

```

from econml.cate_interpreter import SingleTreeCateInterpreter

# est is the fitted CATE model (e.g., from DR-Learner)
# include_model_uncertainty=True uses bootstrap-of-little-bags for CIs
intrp = SingleTreeCateInterpreter(include_model_uncertainty=True, max_depth=2)
intrp.interpret(est, X_test)

# Plot the distillation tree
intrp.plot(feature_names=X_test.columns)

```

3.2 Interpretation Tool 2: SHAP Values

To gain a more granular, feature-level understanding of the CATE model, we can employ **SHAP** (**SH**apley **A**dditive **eX**planations). SHAP is a game-theoretic approach that explains any single prediction as a sum of contributions from each feature, providing both the magnitude and direction of each feature’s influence.

For a single prediction $\hat{\tau}(x_i)$, SHAP decomposes it as:

$$\hat{\tau}(x_i) = \phi_0 + \sum_{j=1}^p \phi_j(x_{ij})$$

where ϕ_0 is the base value (the average CATE prediction) and $\phi_j(x_{ij})$ is the SHAP value for feature

j of individual i . This tells us how much each feature pushed the prediction away from the average.

Applying SHAP to our CATE model allows us to see which features are pushing the predicted treatment effect up or down for each individual. A summary plot (Figure 2) visualizes these effects across the entire dataset.

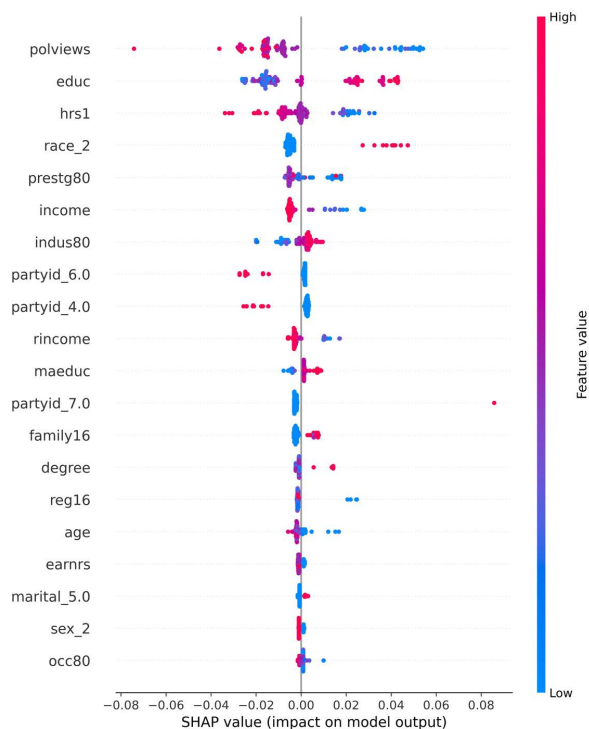


Figure 2: SHAP summary plot for the CATE model in the Welfare Experiment. It confirms that **polviews** and **educ** are the most important features driving heterogeneity. High values of **polviews** (more conservative) are associated with a more negative treatment effect. See Jupyter Notebook.

In the Welfare Experiment, the SHAP summary plot confirms the findings from the distillation tree: **polviews** and **educ** are the two most important features. The plot also provides directionality: high values of **polviews** (more conservative views, shown in red) are associated with a more negative SHAP value, pushing the CATE prediction down. This level of detail is invaluable for building a narrative around the model's predictions.

Distillation Trees vs. SHAP

Use both! Distillation trees provide the **big picture**—the main effects and interactions. SHAP provides the **granular, per-feature story**. Together, they offer a comprehensive interpretation of your black-box CATE model.

Python Pseudocode: SHAP with EconML

EconML estimators have a built-in `.shap_values()` method.

```
import shap

# est is the fitted CATE model
shap_values = est.shap_values(X_test)

# The output is a dictionary, e.g., {'Y': {'T': shap_values_array}}
# We need to access the nested array

# Create the SHAP summary plot
shap.summary_plot(shap_values['Y']['T'], X_test)
```

Important Transition: From Interpretation to Validation

It is crucial to remember that everything in this section is about *interpreting what the model has learned*. It is not a statistical test. The patterns identified by distillation or SHAP are not claimed to be statistically significant. They could be real, or they could be spurious correlations or random noise that the model fit in the training data. The next section is dedicated to the formal statistical tests needed to build confidence that the heterogeneity is real.

4 Statistical Validation: Is the Heterogeneity Real?

Interpretation tells us *what* our model has learned, but it doesn't tell us if what it learned is *true*. The final and most critical step is statistical validation. Can we be statistically confident that our model has captured a genuine signal of heterogeneity, rather than just fitting noise in the training data? Can we trust its recommendations for personalization?

To answer this, we use our final, untouched **validation set**. The framework is as follows:

1. Train nuisance models $(\hat{g}_0, \hat{g}_1, \hat{p})$ on the training data.
2. Use these to compute DR pseudo-outcomes, Y^{DR} , on the validation set.
3. Use the chosen CATE model, $\hat{\tau}(X)$, to make predictions on the validation set.
4. Perform a series of tests comparing the DR pseudo-outcomes to the CATE predictions.

We will cover three complementary tests: the BLP test, GATE analysis, and the Qini curve.

4.1 Test 1: The Best Linear Predictor (BLP) Test

The first and simplest validation test is the Best Linear Predictor (BLP) test. It checks whether our CATE model, $\hat{\tau}(X)$, has captured any genuine signal of treatment effect heterogeneity by running a simple linear regression on the validation set.

The BLP Test

Regress the DR pseudo-outcomes, Y^{DR} , on the demeaned CATE predictions from your model:

$$Y_i^{DR} = \alpha + \beta (\hat{\tau}(X_i) - \bar{\hat{\tau}}) + \epsilon_i$$

where $\bar{\hat{\tau}}$ is the average CATE prediction on the validation set.

Interpreting the Coefficients.

- The intercept, $\hat{\alpha}$, provides an estimate of the Average Treatment Effect (ATE).
- The slope coefficient, $\hat{\beta}$, is the key quantity of interest. It estimates the covariance between the true Individual Treatment Effect (ITE) and our model's predictions, normalized by the variance of our model's predictions:

$$\beta = \frac{\text{Cov}(Y(1) - Y(0), \hat{\tau}(X))}{\text{Var}(\hat{\tau}(X))}$$

Essentially, β measures the correlation between the true heterogeneity and the heterogeneity captured by our model.

Decision Rules.

- If our model $\hat{\tau}(X)$ were perfect (i.e., equal to the true CATE $\tau(X)$), we would expect $\beta = 1$.
- A well-calibrated model should have a 95% confidence interval for $\hat{\beta}$ that **contains 1**.
- To conclude that our model has detected any real heterogeneity, the 95% confidence interval for $\hat{\beta}$ must **exclude 0**.
- If the confidence interval for $\hat{\beta}$ contains 0, our model has failed to find a statistically significant signal of heterogeneity.

In the Welfare Experiment, the BLP test yields $\hat{\beta} = 1.47$ with a 95% CI of $[0.94, 1.99]$. Since the CI excludes 0 and contains 1, we can conclude that the model has successfully detected heterogeneity and is well-calibrated.

	coef	std err	P > z	[0.025	0.975]
const	-0.3839	0.016	0.000	-0.416	-0.352
$\tau_*(X)$	1.4655	0.267	0.000	0.943	1.988

Table 1: BLP results in the welfare experiment. See Jupyter Notebook.

Python Pseudocode: BLP Test

```
import statsmodels.api as sm

# Assume: best_cate_model is the final CATE model from selection step
#         X_val, Y_dr_val are on the validation set
# Predict CATE scores on the validation set
tau_hat_val = best_cate_model.predict(X_val)
tau_hat_demeaned = tau_hat_val - tau_hat_val.mean()
X_reg = sm.add_constant(tau_hat_demeaned) # Add intercept

blp_model = sm.OLS(Y_dr_val, X_reg).fit(cov_type='HCO') # Robust SEs

alpha_hat = blp_model.params[0]
beta_hat = blp_model.params[1]
beta_ci = blp_model.conf_int()[1]

print(f"BLP Beta: {beta_hat:.2f}")
print(f"95% CI: [{beta_ci[0]:.2f}, {beta_ci[1]:.2f}]" )
```

4.2 Test 2: Group Average Treatment Effects (GATEs)

The BLP test provides a single, summary measure of validity. The Group Average Treatment Effect (GATE) analysis offers a more granular and often more intuitive validation. The core idea is to check if the groups of individuals that our model *believes* will have high or low treatment effects actually *do* have statistically different effects when evaluated on the validation data.

Algorithm: GATE Analysis

1. **Define Groups based on CATE Quantiles:** Using the distribution of CATE predictions, $\hat{\tau}(X)$, on the *training* or *scoring* set, we can calculate quantiles $\hat{\mu}(q)$ for various values of q and define quantile-based groups on the validation set. For example, the "bottom quartile" (G_0) could be individuals in the validation set with $\hat{\tau}(X_i) < \hat{\mu}(0.25)$ and the "top quartile" (G_3) could be those with $\hat{\tau}(X_i) > \hat{\mu}(0.75)$, where $\hat{\mu}(q)$ is the empirical q -th percentile of the CATE predictions.
2. **Estimate Group Effects:** For each group, estimate its group average treatment effect (GATE) using the DR estimator on the **validation set**:

$$\widehat{\text{GATE}}(G_k) = \frac{1}{|G_k|} \sum_{i \in G_k} Y_i^{DR}$$

Also compute the 95% confidence interval for each GATE estimate.

3. **Check for Separation:** The key validation check is whether the confidence intervals for the different groups are statistically separated. For instance, we hope to see that the CI for the bottom group lies entirely below the CI for the top group. If so, pause and celebrate! Your model is working.

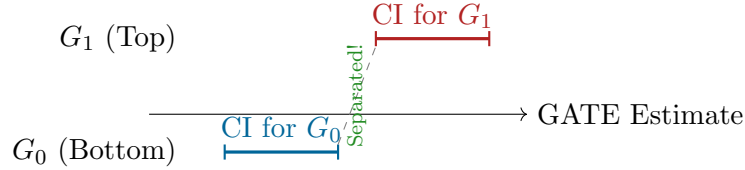


Figure 3: Visual interpretation of a successful GATE analysis. The 95% confidence interval for the GATE of the bottom group is entirely below the CI for the top group, providing strong evidence that the model has identified meaningful heterogeneity.

The binary split into a bottom and top group is a useful starting point, but we can go further. More ambitiously, we can define **quartile groups** (or even finer):

$$\begin{aligned} G_0 &= \{i : \hat{\tau}(X_i) < \hat{\mu}(0.25)\} \\ G_1 &= \{i : \hat{\mu}(0.25) \leq \hat{\tau}(X_i) < \hat{\mu}(0.50)\} \\ G_2 &= \{i : \hat{\mu}(0.50) \leq \hat{\tau}(X_i) < \hat{\mu}(0.75)\} \\ G_3 &= \{i : \hat{\tau}(X_i) \geq \hat{\mu}(0.75)\} \end{aligned}$$

We compute the GATE and 95% CI for each group. Ideally, the GATEs should be **monotonically ordered** (increasing from G_0 to G_3). You can go finer than quartiles if you have enough samples, but be careful about statistical power—with deciles, each group contains only 10% of the data, so

the confidence intervals will be wider.

Python Pseudocode: GATE Analysis

```
# Assume: best_cate_model is the final CATE model from selection step
#         X_train, X_val, Y_dr_val are from the data split
# Predict CATE scores on training set (for quantiles) and validation set
tau_hat_train = best_cate_model.predict(X_train)
tau_hat_val = best_cate_model.predict(X_val)

# 1. Define Groups (e.g., quartiles)
quantiles = np.array([0.25, 0.5, 0.75])
mu_q_hat = np.percentile(tau_hat_train, quantiles * 100)
# np.digitize assigns bins: 0 = below Q25, 1 = Q25-Q50, 2 = Q50-Q75, 3 = above Q75
groups = np.digitize(tau_hat_val, mu_q_hat, right=True)

# 2. Estimate Group Effects
gates, gate_ses = [], []
for k in sorted(np.unique(groups)):
    mask = (groups == k)
    Y_dr_k = Y_dr_val[mask]
    gate_k = np.mean(Y_dr_k)
    se_k = np.std(Y_dr_k) / np.sqrt(len(Y_dr_k))
    gates.append(gate_k)
    gate_ses.append(se_k)
print(f"GATE for Group {k}: {gate_k:.3f} +/- {1.96*se_k:.3f}")
```

4.2.1 The Calibration Plot

We then visualize the GATEs with a **calibration plot**. This plot compares what the model *believed* the average effect would be for a group (the x-axis) against what the DR estimator *found* the effect to be on the validation set (the y-axis). Specifically:

- **X-axis:** The average CATE prediction within each group, $\bar{\hat{\tau}}_{G_k} = \frac{1}{|G_k|} \sum_{i \in G_k} \hat{\tau}(X_i)$.
- **Y-axis:** The DR point estimate of the GATE on the validation set, $\widehat{\text{GATE}}(G_k)$, with 95% CI error bars.

If the model is well-calibrated, the points should lie close to the **45-degree line**—what the model believes matches what the data shows. You can also compute a summary **calibration error**:

$$\text{CalError} = \sum_k \frac{|G_k|}{n} \cdot |\widehat{\text{GATE}}(G_k) - \bar{\hat{\tau}}_{G_k}|$$

where $\bar{\hat{\tau}}_{G_k}$ is the average predicted CATE within group G_k . A lower error is better.

Two Goals of GATEs

When analyzing GATEs, we have two goals. First, we want to see **separation** between the groups (e.g., the CI for the bottom quartile is below the CI for the top quartile). This confirms that we have found statistically significant heterogeneity. Second, we want to see good **calibration** (the GATEs are close to the average predicted CATE for each group). This confirms that the magnitude of our model's predictions is trustworthy.

Python Pseudocode: Calibration Plot

```
# Building on the GATE pseudocode above...

# Calculate average predicted CATE within each group
mean_tau_hat_k = [np.mean(tau_hat_val[groups == k])
                  for k in sorted(np.unique(groups))]

# Create the plot
plt.errorbar(mean_tau_hat_k, gates, yerr=1.96*np.array(gate_ses), fmt='o', capsize=5,
             label='GATEs')

# Plot the 45-degree line
lims = [min(plt.xlim()[0], plt.ylim()[0]), max(plt.xlim()[1], plt.ylim()[1])]
plt.plot(lims, lims, 'k--', alpha=0.75, zorder=0, label='45-degree line')

plt.xlabel("Predicted CATE (Group Average)")
plt.ylabel("Estimated GATE (DR Estimate)")
plt.title("Calibration Plot")
plt.legend()
plt.show()
```

Welfare Experiment: Calibration Plot. In the Welfare Experiment, the calibration plot for the four quartile groups shows that the points lie close to the 45-degree line. The bottom quartile has a predicted average CATE of approximately -0.42 and a DR estimate of approximately -0.51 , while the top quartile has a predicted average CATE of approximately -0.28 and a DR estimate of approximately -0.26 . Crucially, the bottom and top quartiles are **statistically separated**, confirming that the model has identified meaningful heterogeneity.

Characterizing the Groups. Once you've found statistically different groups, you want to understand what makes them different. Two common approaches are:

1. **Difference in means:** For each feature X_j , compute the difference in the average feature value between the top and bottom groups, $\bar{X}_j^{G_1} - \bar{X}_j^{G_0}$. This tells you how the high-responder and low-responder groups differ on each feature.
2. **Classification tree:** Build a dataset with samples from G_0 (label 0) and G_1 (label 1). Train a shallow decision tree classifier on features X . This gives a simple, rule-based char-

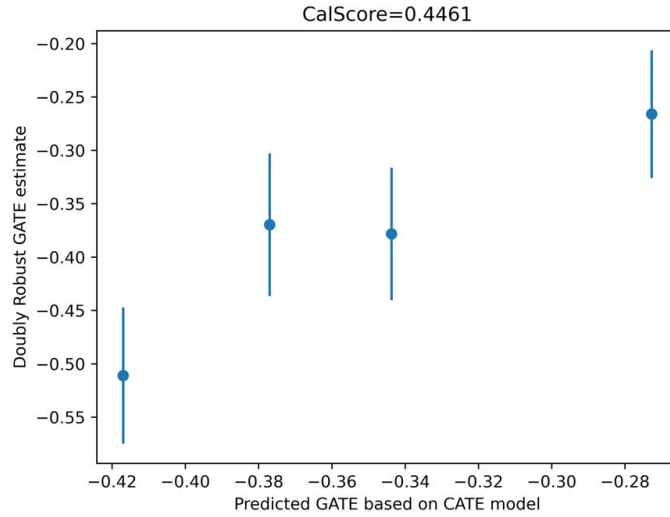


Figure 4: Calibration plot for the Welfare Experiment. The GATEs for the four quartiles are plotted against the average CATE prediction within each quartile. The points lie close to the 45-degree line, and the top and bottom quartiles are statistically separated, indicating a well-calibrated model. See Jupyter Notebook.

acterization of the dominant factors separating the groups. In the Welfare Experiment, this method confirms the distillation tree’s findings, showing that political views, race, and age are significant differentiators between the top and bottom responder groups.

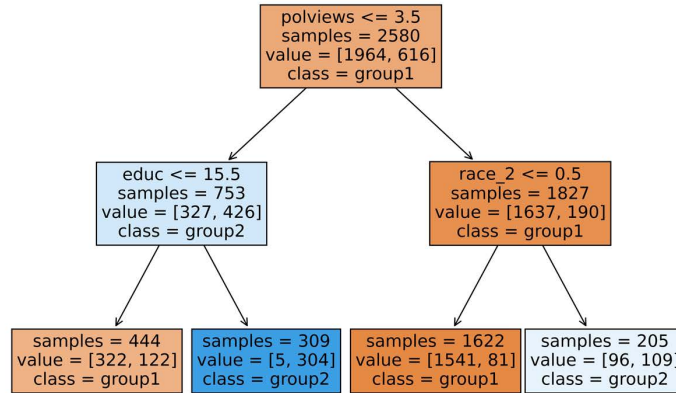


Figure 5: A depth-2 decision tree trained to predict whether an individual belongs to the bottom (group1) or top (group2) quartile of the CATE model’s outputs in the Welfare Experiment. It highlights that political views, education and race are primary drivers of heterogeneity. Notebook.

4.3 Test 3: The Qini Curve

Our final validation tool, the Qini curve, directly addresses the question of policy value: is our CATE model useful for treatment prioritization? It quantifies the incremental benefit of treating the individuals our model ranks highest, compared to a random targeting policy.

The Policy Value Framing. Imagine you have a limited budget and can only treat a fraction, say $\pi \approx 1 - q$, of the population. How do you decide who to treat?

- **CATE-based Policy:** If you trust your CATE model, you should treat the fraction π of the population with the highest predicted CATE scores. This corresponds to the group $G(q) = \{i : \hat{\tau}(X_i) > \hat{\mu}(q)\}$, where $\hat{\mu}(q)$ is the empirical q -th percentile of CATE predictions. The expected total benefit from this policy would be the average treatment effect for this specific group (the GATE) multiplied by the number of people treated: $\pi \cdot N \cdot \text{GATE}(G(q))$, where $\pi = \Pr(\hat{\tau}(X) > \hat{\mu}(q)) \approx 1 - q$.

$$\begin{aligned} \text{Benefit-Priority}(q) &= \mathbb{E}[Y(1) - Y(0) \mid \hat{\tau}(X) > \hat{\mu}(q)] \cdot \Pr(\hat{\tau}(X) > \hat{\mu}(q)) \\ &= \text{GATE}(G(q)) \cdot \Pr(\hat{\tau}(X) > \hat{\mu}(q)) \end{aligned}$$

- **Random Policy:** Alternatively, you could treat a random fraction of the population of the same size. The expected total benefit from this policy would be the average treatment effect (ATE) multiplied by the number of people treated: $\pi \cdot N \cdot \text{ATE}$.

$$\text{Benefit-Random}(q) = \text{ATE} \cdot \Pr(\hat{\tau}(X) > \hat{\mu}(q))$$

The Qini curve measures the *difference* in the average value generated by these two policies, scaled by the fraction treated. It tells us the **gain** from using our model for prioritization.

Definition: The Qini Curve

The **Qini curve** at q measures the *gain* from CATE-based targeting over random targeting:

$$\begin{aligned} \tau_{\text{Qini}}(q) &= \text{Benefit-Priority}(q) - \text{Benefit-Random}(q) \\ &= \Pr(\hat{\tau}(X) > \hat{\mu}(q)) \cdot (\text{GATE}(G(q)) - \text{ATE}) \end{aligned}$$

Derivation from Policy Value. The value of the Qini curve arises naturally from this comparison. The gain per person is the difference in the average effects, $\text{GATE}(G(q)) - \text{ATE}$. The total gain is this difference multiplied by the number of people we apply this policy to, which is the fraction $\pi = \Pr(\hat{\tau}(X) > \hat{\mu}(q))$.

Key Insight: Re-Interpreting The Qini Curve

The Qini curve at q can be shown to be equal to the covariance of the true ITE with the indicator of your model believing the CATE is above the q -th percentile:

$$\tau_{\text{Qini}}(q) = \text{Cov}(Y(1) - Y(0), \mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\})$$

This can be identified and estimated by the DR analogue on the validation set:

$$\hat{\tau}_{\text{Qini}}(q) = \widehat{\text{Cov}}_n(Y^{DR}, \mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\})$$

Derivation of the Qini Covariance Formula

Let $\mathcal{E}_q = \mathbf{1}\{\hat{\tau}(X) > \hat{\mu}(q)\}$ and $\pi_q = \Pr(\hat{\tau}(X) > \hat{\mu}(q))$ and let $\text{ITE} = Y(1) - Y(0)$ be the random variable corresponding to a unit's effect. The Qini value is defined as the gain from the targeted policy:

$$\begin{aligned}\tau_{\text{Qini}}(q) &= \pi_q \cdot (\mathbb{E}[Y(1) - Y(0) \mid \hat{\tau}(X) > \hat{\mu}(q)] - \mathbb{E}[Y(1) - Y(0)]) \\ &= \pi_q \cdot \mathbb{E}[\text{ITE} \mid \hat{\tau}(X) > \hat{\mu}(q)] - \pi_q \cdot \mathbb{E}[\text{ITE}] \\ &= \mathbb{E}[\text{ITE} \cdot \mathcal{E}_q] - \pi_q \cdot \mathbb{E}[\text{ITE}] \quad (\text{since } \pi_q = \mathbb{E}[\mathcal{E}_q]) \\ &= \mathbb{E}[\text{ITE} \cdot \mathcal{E}_q] - \mathbb{E}[\mathcal{E}_q] \cdot \mathbb{E}[\text{ITE}] \\ &= \text{Cov}(\text{ITE}, \mathcal{E}_q)\end{aligned}$$

This shows that the value of the targeting policy is exactly the covariance between the true treatment effect and the indicator of being in the targeted group.

Qini Curve vs. BLP

The BLP test measures the correlation between the true ITE and the *value* of the CATE prediction. The Qini curve measures the covariance between the true ITE and a *non-linear transformation* of the prediction (the indicator of being in a top quantile). This gives the Qini curve a more direct policy-prioritization interpretation.

Plotting and Interpreting the Qini Curve. The Qini curve is plotted with the fraction of the population treated on the x-axis and the Qini value on the y-axis. It measures the cumulative gain from the policy.

- If the curve and its confidence bands are consistently **above zero**, it means our CATE model provides a more valuable targeting policy than random assignment.
- The peak of the curve indicates the optimal fraction of the population to treat to maximize the total value generated by the policy. As the fraction treated approaches 0 or 1, the curve

must return to 0, since treating no one or everyone removes any gain from prioritization.

- A flat curve at zero indicates that the model provides no value for prioritization.

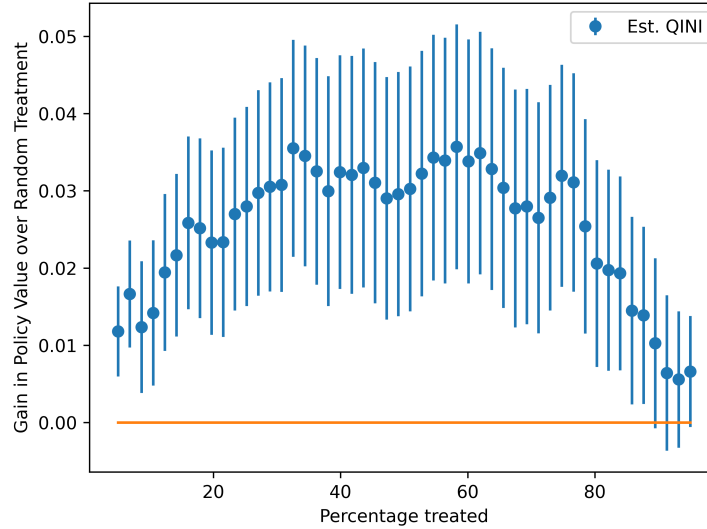


Figure 6: The Qini curve for the Welfare Experiment. The curve is consistently above zero, indicating that the CATE model is effective for prioritizing who to show the “welfare” framing to in order to maximize the total effect. In this context, prioritizing into treatment is not a relevant policy decision, as the treatment is not costly. Even in such settings, the Qini curve can be seen as a statistical validator of the model’s learned effect heterogeneity. See Jupyter Notebook.

Python Pseudocode: Qini Curve

```
# Assume: best_cate_model is the final CATE model from selection step
#         X_train, X_val, Y_dr_val are from the data split
# Predict CATE scores on training set (for quantiles) and validation set
tau_hat_train = best_cate_model.predict(X_train)
tau_hat_val = best_cate_model.predict(X_val)

q_grid = np.linspace(0.05, 0.95, 50)
cutoffs = np.percentile(tau_hat_train, q_grid * 100)
qini, se, frac = np.zeros(50), np.zeros(50), np.zeros(50)
for i, c in enumerate(cutoffs):
    ind = (tau_hat_val > c).astype(float)
    frac[i] = np.mean(ind)
    AB = (Y_dr_val - Y_dr_val.mean()) * (ind - frac[i])
    qini[i] = np.mean(AB)
    se[i] = np.std(AB) / np.sqrt(len(Y_dr_val))

plt.fill_between(frac, qini - 1.96*se, qini + 1.96*se, alpha=0.3)
plt.plot(frac, qini); plt.axhline(y=0, color='red', ls='--')
```

4.4 All-in-One Validation with EconML's DRTester

EconML's DRTester automates the entire validation pipeline (BLP, GATEs, Qini, TOC) in a few lines.

Python Pseudocode: DRTester

```
from econml.validate.drtester import DRTester

# Step 1: Initialize with nuisance models and CATE model
tester = DRTester(
    model_regression=GradientBoostingRegressor(),
    model_propensity=RandomForestClassifier(),
    cate=est # any fitted CATE estimator
)

# Step 2: Fit nuisance models on train, evaluate on val
tester.fit_nuisance(Xval, Dval, Yval, Xtrain, Dtrain, Ytrain)

# Step 3: Run all validation tests at once
res = tester.evaluate_all(Xval, Xtrain)

# Step 4: View the summary table and plots
res.summary()
res.plot_cal(tmt=1)

# the keyword ucb dictates uniform confidence bands
# and not just pointwise confidence intervals across values of q
res.plot_qini(tmt=1, err_type='ucb2')
```

Together, the BLP test, GATE analysis, and Qini curve form a robust suite of validation tools that allow us to rigorously assess the quality and reliability of our selected CATE model from multiple perspectives.

5 In-Class Activity: Applying CATE Analysis to Digital Health

To solidify the concepts of CATE model selection, interpretation, and validation, we will work through a case study based on a real-world research paper. This activity challenges us to apply the entire workflow we have learned to a practical problem in digital health.

5.1 The Scenario: Digital Health Interventions for Youth with Diabetes

Case Study Scenario

A children’s hospital utilizes a **remote patient monitoring (RPM)** application to assist young patients with type 1 diabetes in managing their blood glucose levels. Through this app, clinicians can deliver personalized **digital health interventions (DHIs)**—such as motivational messages, reminders, educational content, and behavioral nudges.

The central research question for the hospital is: **Which patients benefit most from these interventions, and when?**

This is a CATE problem. The hospital wants to move beyond a one-size-fits-all approach and develop a personalized policy for sending messages to maximize positive impact on patient health.

5.2 The Data and the Challenge

The available dataset contains the following information for each patient on each day:

- **Patient Features (X):** A rich set of covariates including demographics (age, gender), clinical history, baseline glycemic control (e.g., average glucose, time-in-range from continuous glucose monitoring data), and measures of patient engagement with the app.
- **Intervention:** A binary indicator for whether a clinician sent a message to the patient on that day.
- **Message Content:** The raw text of the message that was sent.
- **Outcome (Y):** A measure of glycemic control, such as the patient’s time-in-range for blood glucose, in the days following the potential intervention.

5.3 Group Task and Discussion

In breakout groups, the task is to devise a complete analysis plan to address the hospital’s research question. This involves framing the problem, outlining a technical solution, and considering the practical aspects of interpretation and deployment.

Part 1: Framing the HTE Problem.

- How would you formally define the treatment D , outcome Y , and covariates X ?
- What is the precise definition of the CATE, $\tau(x)$, in this context?
- A crucial modeling choice is how to represent the treatment. Is a simple binary indicator for receiving *any* message sufficient? How could you leverage the **message content** to create a richer, more informative representation of the treatment? This is a prompt to think about the

role of Natural Language Processing (NLP) in creating feature vectors (embeddings) from the text of the messages. This transforms the problem from a binary treatment to a continuous or multi-dimensional treatment setting, which can be handled by a generalized R-Learner.

Part 2: Devising a Solution Plan.

- Which meta-learners would you consider, and why? Given the observational nature of the data, which learner seems most appropriate?
- How would you implement the train/score/validate data splitting strategy for model selection?
- Which validation tests (BLP, GATEs, Qini) would you prioritize to build confidence in your final model?

Part 3: Interpretation and Deployment.

- How would you explain your CATE model's findings to the clinicians who will be using it? Which interpretation tools (distillation trees, SHAP) would be most effective for communicating the results to a non-technical audience?
- What would a real-world deployment of your CATE model look like? Would it be a fully automated system, or a decision-support tool for clinicians?

5.4 Key Challenges

Key Challenge 1: Observational Data

This is an **observational study**. Clinicians decide when and to whom they send messages based on their expert judgment, patient needs, and recent health data. The treatment (receiving a message) is **not randomly assigned**. This means that there is a high risk of confounding; for example, clinicians might be more likely to message patients who are already doing poorly, which could create a spurious negative correlation between receiving a message and subsequent health outcomes. This challenge is addressed by controlling for all the factors that the physician sees on the app before deciding whether to send a message and what message to send, which include high-dimensional time-series data and text representations of patient state, thus requiring representation learning on the control variables to make them suitable for the methods we've discussed.

Key Challenge 2: Treatment Representation

The intervention is described as a whole message, which does not immediately fit the paradigms we've discussed (e.g., categorical treatment or continuous treatment). This requires preprocessing and representation learning on the treatment (text message) to transform it into a form that can be ingested by all the methods we've discussed.

Therefore, any valid causal analysis must use methods that can adjust for high-dimensional controls in a flexible manner, such as the DR-Learner, while incorporating appropriate feature engineering and representation learning for both the treatment and the controls.

5.5 What the Paper Did: Clinician-Informed Representations

This case study is based on the research paper **”Learning Explainable Treatment Policies with Clinician-Informed Representations”** by Ferstad, Fox, Scheinker, and Johari (2024). The authors’ key insight was that the *content* of the message is a critical component of the treatment itself, and how this content is represented is paramount for learning an effective and interpretable policy.

The Role of Messages and NLP. The authors recognized that treating all messages as a single, binary treatment is a gross oversimplification. The text of the message is a rich source of information. The authors performed clustering of the treatments in various types based on the contents of the message. This clustering then turns the problem back into a categorical treatment problem, where the goal is to understand the effect of each type of message (where the type of message is characterized by the cluster to which it belongs). Once framed this way, all the tools that we have learned in today’s lecture are applicable to learn CATEs for each type of message, interpret them, and validate them statistically.

Clinician-Informed vs. Black-Box Representations. The paper makes a key distinction between two approaches to representing and clustering the treatment messages:

- **Black-Box Representations:** This approach uses raw NLP embeddings of messages directly and then clusters the messages in representation space. While these embeddings are rich and high-dimensional, they are also difficult for a clinician to interpret. A policy that recommends treating a patient because their feature vector is close to some opaque point in a high-dimensional embedding space is unlikely to be trusted or adopted.
- **Clinician-Informed Representations:** In this approach, the authors worked directly with clinicians to define a small number of meaningful, high-level categories for the messages (e.g., ”motivational,” ”educational,” ”logistical reminder”). They then used these categories as the features to represent the treatment. This low-dimensional, interpretable representation is much easier for clinicians to understand and act upon.

The paper’s main finding is that the policies learned from the **clinician-informed representations** were significantly more effective and efficient than those learned from the black-box embeddings. This highlights a crucial lesson: in applied causal ML, technical sophistication must be balanced with domain knowledge and the practical need for interpretability. The best model is not always the most complex one, but the one that best integrates statistical rigor with real-world

context. The paper’s authors also made a strong argument for unconfoundedness in this remote monitoring setting: since the clinician’s decisions are based only on data visible within the app (which is all recorded and can be controlled for), the risk of unmeasured confounding is low.

This case study serves as a perfect illustration of the entire workflow discussed in this chapter. The authors had to estimate the CATE in an observational setting (requiring methods like the DR-Learner), select from multiple candidate models (requiring a principled selection metric like the DR-Loss), validate their final model’s performance (using tests like GATEs), and interpret the results to provide actionable insights for clinicians.

6 Summary: The Complete CATE Workflow

This chapter has provided a comprehensive, end-to-end workflow for applied CATE analysis. The key steps are:

1. **Train:** Fit multiple meta-learners (S, T, DR, R) with various underlying ML methods. Within each family, tune hyperparameters using the learner’s native loss function (e.g., using an AutoML tool like FLAML).
2. **Select:** Use the **DR-Loss**, evaluated on a held-out scoring set, as the ultimate metric to select the single best CATE model from the pool of tuned candidates, or to create a stacked ensemble. In RCTs, this provides an unbiased estimate of the true CATE MSE.
3. **Interpret:** Employ tools like **distillation trees** to get a high-level, global understanding of the model’s logic, and use **SHAP values** to obtain granular, per-feature explanations for individual predictions.
4. **Validate:** On a final, held-out validation set, perform a suite of statistical tests to build confidence in the model:
 - **BLP test:** Check if the model has captured any statistically significant signal of heterogeneity.
 - **GATE analysis:** Verify that the model can identify subgroups with statistically different treatment effects, often visualized with a calibration plot.
 - **Qini curve:** Assess the model’s practical value for treatment prioritization and resource allocation.

By following this disciplined process of training, selecting, interpreting, and validating, we can move from a zoo of potential models to a single, reliable, and actionable CATE model that can be deployed with confidence to power personalized decision-making.

References

- [1] Chernozhukov, V., Demirer, M., Duflo, E., & Fernández-Val, I. (2025). Fisher–schultz lecture: Generic machine learning inference on heterogeneous treatment effects in randomized experiments, with an application to immunization in india. *Econometrica*, 93(4), 1121-1164.
- [2] Dwivedi, R., Tan, Y. S., Park, B., Wei, M., Horgan, K., Madigan, D., & Yu, B. (2020). Stable discovery of interpretable subgroups via calibration in causal studies. *International Statistical Review*, 88, S135-S178.
- [3] Ferstad, J. O., Fox, E. B., Scheinker, D., & Johari, R. (2024). Learning Explainable Treatment Policies with Clinician-Informed Representations: A Practical Approach. *arXiv preprint arXiv:2411.17570*.
- [4] Foster, D. J., & Syrgkanis, V. (2023). Orthogonal statistical learning. *The Annals of Statistics*, 51(3), 879-908.
- [5] Nie, X., & Wager, S. (2021). Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2), 299-319.
- [6] Yadlowsky, S., Fleming, S., Shah, N., Brunskill, E., & Wager, S. (2025). Evaluating treatment prioritization rules via rank-weighted average treatment effects. *Journal of the American Statistical Association*, 120(549), 38-51.

A Appendix: The Cross (X)-Learner

The X-Learner, or Cross-Learner, is a meta-learner specifically designed to perform well in settings where there is a substantial imbalance in the treatment assignment—that is, when the propensity score $\mu(x) = P(D = 1|X = x)$ is often close to 0 or 1. This is a common scenario in digital experimentation or medical trials with rare diseases, where the treated group is much smaller than the control group. Unlike the DR-Learner, which is motivated by double robustness, the X-Learner is motivated by accuracy and efficiency in these covariate-shift scenarios.

The Core Idea: Leveraging Imputed Effects

The X-Learner’s strategy is built on a clever insight. We can estimate the CATE, $\tau(x)$, in two different ways by imputing the counterfactual outcomes for each group:

1. **For the treated group ($D = 1$):** We use a model trained on the control group, $\hat{g}_0(x) = \mathbb{E}[Y|X = x, D = 0]$, to impute their missing counterfactuals. This gives an estimate of the individual treatment effect for treated units:

$$\widehat{\text{ITE}}_T = Y - \hat{g}_0(x) \quad \text{for units with } D = 1$$

We can then learn the Conditional Average Treatment Effect on the Treated (CATT) by regressing these ITEs on X , i.e. $\hat{\tau}_1(x) = \mathbb{E}[\widehat{\text{ITE}}_T \mid X = x, D = 1]$.

2. **For the control group ($D = 0$):** We use a model trained on the treated group, $\hat{g}_1(x) = \mathbb{E}[Y \mid X = x, D = 1]$, to impute their missing counterfactuals. This gives an estimate of the individual treatment effect for the control units:

$$\widehat{\text{ITE}}_C(x) = \hat{g}_1(x) - Y \quad \text{for units with } D = 0$$

We can then learn the Conditional Average Treatment Effect on the Control (CATC) by regressing these ITEs on X , i.e. $\hat{\tau}_0(x) = \mathbb{E}[\widehat{\text{ITE}}_C \mid X = x, D = 0]$.

Under unconfoundedness, both CATT and CATC are valid estimates of the true CATE, $\tau(x)$. The X-Learner then combines them in a principled, propensity-weighted manner. If we have more treated individuals in some region of x (i.e., $p(x)$ is high), then we have greater trust in the nuisance model $\hat{g}_1$ rather than in the model $\hat{g}_0$. Hence, we will trust the CATC more, $\hat{\tau}_0$, since it only uses $\hat{g}_1$. Analogously, if we have more control individuals in some region of x (i.e., $p(x)$ is low), then we trust the nuisance model $\hat{g}_0$ more rather than the model $\hat{g}_1$. Hence, we will trust the CATT more, $\hat{\tau}_1$, since it only uses $\hat{g}_0$.

The X-Learner Algorithm

1. **Stage 1: Outcome Models.** Train two separate response models: $\hat{g}_1(x)$ on the treated group and $\hat{g}_0(x)$ on the control group.
2. **Stage 2: Imputed Effects.** Impute the treatment effects for each group as described above, yielding $\widehat{\text{ITE}}_T = Y - \hat{g}_0(X)$ for the treated and $\widehat{\text{ITE}}_C = \hat{g}_1(X) - Y$ for the controls.
3. **Stage 3: CATE Regressions.** Train two new models to predict the imputed effects based on the covariates:
 - $\hat{\tau}_1(x) = \mathbb{E}[\widehat{\text{ITE}}_T \mid X = x, D = 1]$
 - $\hat{\tau}_0(x) = \mathbb{E}[\widehat{\text{ITE}}_C \mid X = x, D = 0]$
4. **Stage 4: Weighted Combination.** Combine the two CATE estimates using a propensity score model, $\hat{p}(x) = P(D = 1 \mid X = x)$:

$$\hat{\tau}(x) = \hat{p}(x)\hat{\tau}_0(x) + (1 - \hat{p}(x))\hat{\tau}_1(x)$$

When to Use the X-Learner

The X-Learner shines when the CATE function $\tau(x)$ is much simpler than the response surfaces $g_1(x)$ and $g_0(x)$, and when treatment is imbalanced. In these cases, it can be more accurate than the T-Learner (which struggles to model the response surface for the smaller group) and the R-Learner (whose residual-on-residual approach can be high-variance).